



Univerzitet u Novom Sadu
Fakultet tehničkih nauka

Projektna dokumentacija

GqmPlus – informacioni sistem za upravljanje strateškim ciljevima zasnovan na GQM+Strategies metodologiji

Profesor: prof. dr Vladimir Mandić

Asistent: Andrej Katin

Predmet: Upravljanje razvojem
informacionih sistema

Studenti: Ljuban Opačić IT2-2022

Marko Anić IT36-2022

Teodora Ćorović IT16-2022

Viktor Barić IT12-2022

Nemanja Savković IT63-2022

Anđela Mrđa IT7-2022

Novi Sad, 2026.

Sadržaj:

1. Uvod.....	4
1.1. Opis problema i motivacija za razvoj sistema.....	4
1.2. Ciljevi sistema.....	4
1.3. Obim sistema.....	5
2. Korisničke uloge i permisije.....	5
2.1. RBAC model.....	5
2.2. Definisane permisije.....	6
2.3. Tok autentifikacije i autorizacije.....	6
3. Funkcionalni zahtevi.....	7
3.1. Upravljanje korisnicima i ulogama.....	7
3.2. Upravljanje organizacijama i departmanima.....	7
3.3. Upravljanje strateškim ciljevima.....	7
3.4. Upravljanje strategijama i uticajima na ciljeve.....	8
3.5. Upravljanje premisama.....	8
3.6. GQM ciljevi, pitanja, ciljne vrednosti i merenja.....	8
3.7. Procene verovatnoće ostvarenja.....	8
3.8. Orkestracija distribuiranih transakcija.....	9
3.9. Audit log.....	9
4. Nefunkcionalni zahtevi.....	9
4.1. Bezbednost.....	9
4.2. Skalabilnost.....	9
4.3. Dostupnost i otpornost na greške.....	9
4.4. Održivost i razvojni standardi.....	10
4.5. Praćenje i dijagnostika.....	10
5. Arhitektura sistema.....	10
5.1. Opšti pregled arhitekture.....	10
5.2. Mikroservisni principi.....	11
5.3. Database-per-service pattern.....	11
5.4. Interna slojevita arhitektura servisa.....	11
6. Opis mikroservisa.....	12
6.1. User Service.....	12
6.2. Department Service.....	13
6.3. Goal Service.....	13
6.4. Premise Service.....	15
6.5. GQM Goal Service.....	15
6.6. Assessment Service.....	16
6.7. Orchestration Service.....	16
6.8. Audit Service.....	17
7. Dijeljene biblioteke.....	18
7.1. Shared.HMAC.....	18

7.2. Shared.Auth.....	18
7.3. Shared.Contracts.....	19
7.4. Shared.ErrorHandling.....	19
8. Opis baze podataka.....	20
8.1. Database-per-service pristup.....	20
8.2. Entity Framework Core konfiguracija.....	20
8.3. Migracije i seed podaci.....	21
8.4. Volumi za trajnost podataka.....	21
9. Komunikacija između servisa.....	21
9.1. HTTP klijenti i IHttpConnectionFactory obrazac.....	21
9.2. HMAC autentifikacija međuservisnih poziva.....	22
9.3. Korelacioni ID i praćenje zahteva.....	22
9.4. Saga obrazac i kompenzacione transakcije.....	22
9.5 RabbitMQ.....	23
10. API Gateway.....	23
10.1. Nginx kao reverzni proksi.....	23
10.2. URL rutiranje.....	23
10.3. Prosleđivanje zaglavlja.....	24
10.4. Generisanje korelacionog ID-a.....	24
11. Frontend aplikacija.....	24
11.1. Arhitektura Angular aplikacije.....	24
11.2. Rutiranje i lazy loading.....	25
11.3. Autentifikacija i upravljanje stanjem.....	25
11.4. API servisi i modeli.....	26
11.5. Interceptor.....	26
11.6. Zaštita ruta.....	27
12. Frontend-backend interakcija.....	27
12.1. Tok komunikacije.....	27
12.2. Format zahteva i odgovora.....	28
12.3. Rukovanje greškama.....	28
13. DevOps postavka.....	28
13.1. Docker i kontejnerizacija.....	28
13.2. Docker Compose orkestracija.....	28
13.3. CI/CD pipeline — GitHub Actions.....	29
13.4. Strategija grananja i verziona kontrola.....	29
13.5. Prometheus.....	30
13.6. Grafana.....	30
13.7. Jaeger.....	30
14. Testiranje.....	30
14.1. Strategija testiranja.....	30
14.2. Unit testovi servisa.....	31

14.3. Testovi validatora.....	31
14.4. Testovi domenskih komponenti.....	31
14.5. E2E testiranje.....	31
14.6. Automatizovano pokretanje u CI.....	31
15. Zaključak.....	32

1. Uvod

1.1. Opis problema i motivacija za razvoj sistema

U savremenim organizacijama jedan od ključnih izazova jeste uspostavljanje jasne i merljive veze između strateških ciljeva organizacije i konkretnih aktivnosti koje se preduzimaju na operativnom nivou. Organizacije često definišu visokoapstraktne ciljeve koji se teško mogu pratiti, kvantifikovati i procenjivati. Posledica toga je nemogućnost pravovremenog reagovanja u slučaju odstupanja od željenih rezultata, kao i nedostatak transparentnosti u procesu upravljanja ciljevima. GQM+ Strategies (Goal-Question-Metric) pristup predstavlja metodologiju koja omogućava sistematično razlaganje strateških ciljeva u merljive indikatore. Metodologija polazi od pretpostavke da svaki cilj treba biti podržan skupom pitanja koja opisuju u kojoj meri se cilj ostvaruje, a svako od tih pitanja treba biti pokriveno odgovarajućim metrikama i konkretnim merenjima. Pored ciljeva, sistem uzima u obzir i premise — pretpostavke i kontekstualne faktore koji utiču na ostvarivost ciljeva.

Ni jedna od dostupnih softverskih alatki ne implementira u celosti GQM+ Strategies metodologiju na način koji odgovara potrebama većih organizacija sa složenom hijerarhijskom strukturom. Razvoj namenskog sistema koji podržava ceo životni ciklus strateškog cilja — od definisanja, prek praćenja i merenja, do ocene verovatnoće ostvarenja — predstavlja rešenje koje direktno adresira ovaj nedostatak.

1.2. Ciljevi sistema

Sistem GqmPlus razvijen je sa ciljem da organizacijama pruži centralizovanu platformu za upravljanje strateškim ciljevima u skladu sa GQM+ Strategies metodologijom. Primarni cilj sistema je da omogući korisnicima definisanje strateških ciljeva sa svim relevantnim atributima, uključujući fokus cilja, objekat, veličinu promene, ograničenja i vremenski okvir važenja. Uz to, sistem treba da podrži razlaganje svakog cilja na strategije, a strategija na pitanja, ciljne vrednosti i merenja, čime se uspostavlja kompletna merljiva hijerarhija.

Dodatni ciljevi sistema obuhvataju: upravljanje premisama koje definišu kontekstualne pretpostavke ciljeva i strategija, praćenje verovatnoća ostvarenja ciljeva kroz procene, upravljanje organizacionom strukturom kroz organizacije i departmane, kao i obradu multi-tenant scenarija u kojima jedan korisnik može imati različite uloge u različitim organizacijama. Bezbednost i revizija operacija ostvareni su kroz centralizovani audit log koji beleži sve izmene u sistemu.

1.3. Obim sistema

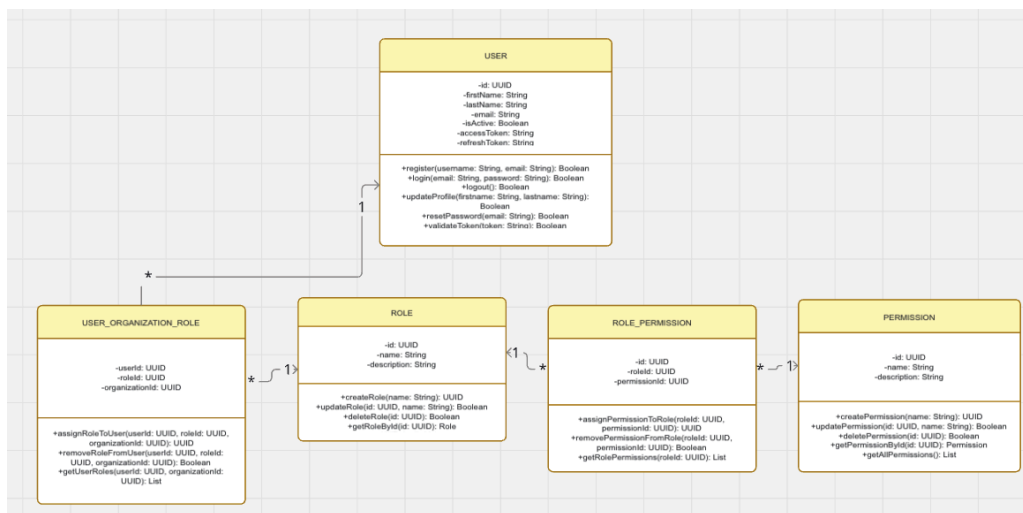
Sistem GqmPlus je namenjen organizacijama koje primenjuju ili planiraju primenu GQM+ Strategies metodologije u svom upravljačkom procesu. Sistem pokriva sledeće oblasti: upravljanje korisnicima, ulogama i permisijama; upravljanje organizacijama i departmanima; celokupan životni ciklus strateških ciljeva i strategija; definisanje i praćenje uticaja na ciljeve; upravljanje premisama; GQM hijerarhiju (ciljevi, pitanja, ciljne vrednosti, merenja); procene verovatnoće; orkestraciju složenih distribuiranih operacija i centralizovano logovanje revizije.

Sistem je dizajniran kao mikroservisna platforma sa odvojenim Angular frontendom koji komunicira sa backendom isključivo kroz Nginx API Gateway. Svaki funkcionalni segment sistema implementiran je kao zasebni, nezavisno rasporedivi mikroservis sa sopstvenom bazom podataka.

2. Korisničke uloge i permisije

2.1. RBAC model

Sistem implementira model kontrole pristupa zasnovan na ulogama (Role-Based Access Control, RBAC). Upravljanje ulogama i permisijama u potpunosti je centralizovano unutar User mikroservisa (*Slika 1*). Uloge su entiteti koji sadrže kolekciju permisija, a korisnicima se dodeljuju putem entiteta UserOrganizationRole, što znači da jedan korisnik može istovremeno imati različite uloge u različitim organizacijama. Ovaj pristup omogućava implementaciju multi-tenant scenarija u kojima organizacioni kontekst određuje skup dozvola koje korisnik poseduje.



Slika 1: ER dijagram entiteta User, Role, Permission, RolePermission, UserOrganizationRole sa vezama

Kada se korisnik prijavi na sistem, korisnički servis izdaje JWT (JSON Web Token) koji sadrži relevantne tvrdnje (claims) korisnika, uključujući identifikator korisnika i permisije. Token se

koristi za zaštitu API endpoint-a na svim mikroservisima, koji samostalno vrše verifikaciju tokena bez oslanjanja na centralni servis za autorizaciju pri svakom zahtevu.

2.2. Definisane permisije

U sistemu su definisane sledeće permisije, koje su usklađene između backend konstanti (UserService.Domain.Constants.Permissions) i frontend modela (auth.models.ts):

Permisija *manage_organizations* daje korisniku pravo na kreiranje, izmenu i brisanje organizacija. Permisija *manage_users* omogućava upravljanje korisničkim nalogima u okviru organizacije. Permisija *manage_roles* daje pravo na definisanje i izmenu uloga i dodelu permisija. Permisija *manage_departments* pokriva upravljanje departmanima unutar organizacije, dok *view_all_departments* omogućava pregled svih departmana bez prava izmene.

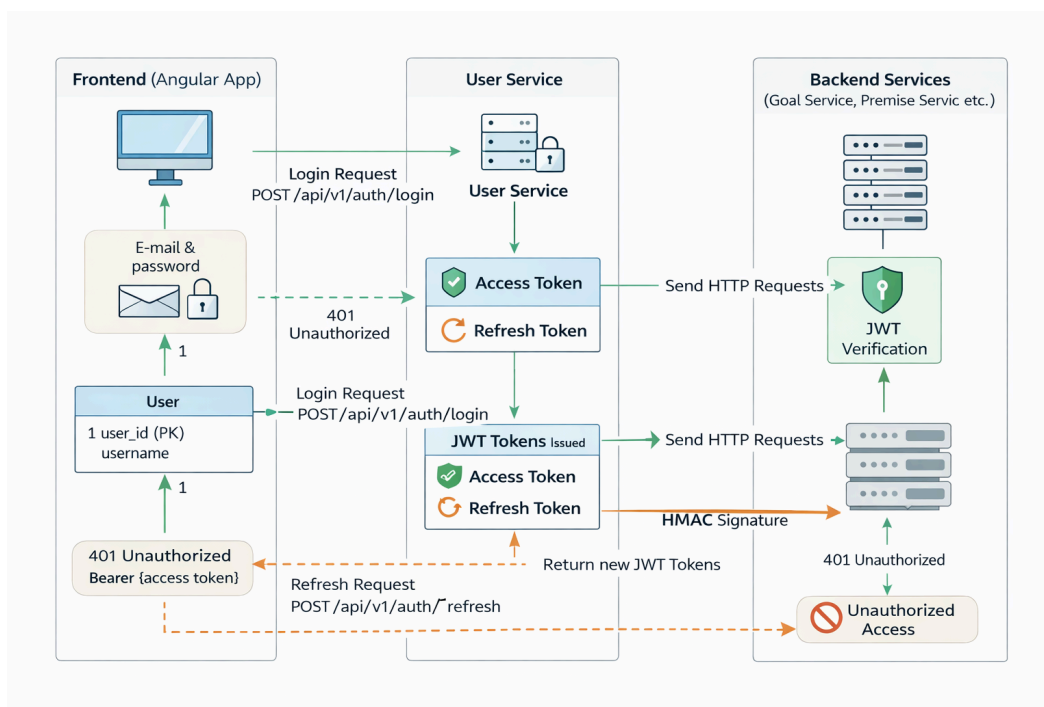
Za strateške ciljeve definisane su permisije *create_goals*, *edit_goals*, *delete_goals* i *view_goals*, koje granularno kontrolišu pristup operacijama nad ciljevima. Permisija *manage_goal_influences* daje pravo upravljanja vezama između ciljeva i strategija. Permisija *record_measurements* omogućava beleženje merenja na ciljnim vrednostima, dok *manage_probability_assessments* daje pravo kreiranja i izmene procena verovatnoće. Permisija *view_analytics* omogućava pristup analitičkim prikazima.

2.3. Tok autentifikacije i autorizacije

Proces autentifikacije korisnika odvija se kroz sledeće korake (*Slika 2*). Korisnik šalje kredencijale (e-mail i lozinku) na endpoint *POST /api/v1/auth/login*. User servis verifikuje kredencijale i, u slučaju uspešne autentifikacije, generiše par tokena: kratkotrajeći **access token** (podrazumevano 15 minuta) i dugotrajniji **refresh token**. Access token se potpisuje tajnim ključem i sadrži identifikator korisnika i ostale relevantne tvrdnje.

Frontend čuva tokene u localStorage i dodaje access token na svaki HTTP zahtev ka backendu kao *Authorization: Bearer {token}* zaglavlje, što je implementirano kroz Angular HTTP interceptor. Kada access token istekne, error interceptor hvatanjem 401 greške inicira poziv *POST /api/v1/auth/refresh* sa refresh tokenom, čime dobija novi par tokena bez potrebe za ponovnim prijavljivanjem korisnika.

Na backendu, svaki servis (osim Orchestration i Audit servisa koji nisu direktno dostupni korisnicima) verifikuje JWT token lokalno, bez mrežnog poziva ka User servisu. Pored JWT autentifikacije korisnika, svaki međuservisni HTTP poziv je zaštićen HMAC potpisom koji garantuje autentičnost i integritet zahteva između servisa.



Slika 2: Dijagram toka autentifikacije

3. Funkcionalni zahtevi

3.1. Upravljanje korisnicima i ulogama

Sistem mora omogućiti registraciju i upravljanje korisničkim nalogima. Svaki korisnik ima atribute: ime, prezime, e-mail adresu, lozinku (čuvanu u hashovanom obliku) i status aktivnosti. Administrator može kreirati korisničke naloge, menjati korisničke podatke i deaktivirati naloge. Sistem mora podržati dodelu i oduzimanje uloga korisnicima u okviru konkretnih organizacija. Jedno korisničko prijavljivanje daje pristup svim organizacijama u kojima korisnik ima ulogu, pri čemu se organizacioni kontekst prenosi kroz X-Organization-Id zaglavlje.

3.2. Upravljanje organizacijama i departmanima

Sistem mora podržati CRUD operacije za organizacije i departmane. Organizacija predstavlja krovni entitet koji može sadržati više departmana. Svaki departman vezan je za tačno jednu organizaciju i može imati menadžera (koji je korisnik sistema). Pregled organizacija i departmana ograničen je permisijama korisnika.

3.3. Upravljanje strateškim ciljevima

Sistem mora omogućiti kreiranje strateških ciljeva sa sledećim atributima: *fokus cilja* (šta se meri), *objekat* (nad čim se meri), *veličina promene* (u kom smeru), *ograničenja* (kontekstualne granice), *status* (Active, OnHold, Completed, Cancelled), *period važenja* (activeFrom, activeTo) i

početna verovatnoća ostvarenja (baselineProbability). Svaki cilj mora biti vezan za departmanom.

Sistem mora podržati promene statusa cilja uz primenu poslovnih pravila o dozvoljenim tranzicijama statusa. Korisnik mora moći da pregleda sve ciljeve dostupne u okviru organizacije i da pristupi detaljnom prikazu koji agregira sve povezane podatke: strategiju, premise, GQM ciljeve i procene.

3.4. Upravljanje strategijama i uticajima na ciljeve

Svaki strateški cilj može imati pridruženu strategiju koja opisuje pristup ostvarenju cilja. Strategija ima attribute: naziv, opis, tip razlaganja (AND — sve podstrategije moraju biti ispunjene, ili OR — dovoljna je jedna) i opcijski referencu na matičnu strategiju (originStrategyId) koja omogućava hijerarhijsko grananje strategija.

Sistem mora podržati i upravljanje uticajima na ciljeve (GoalInfluence), koji opisuju na koji način jedna strategija utiče na određeni cilj, sa opisom prirode uticaja.

3.5. Upravljanje premisama

Sistem mora podržati kreiranje i upravljanje premisama koje definišu pretpostavke i kontekstualne faktore ciljeva. Premisa ima tip (Assumption — pretpostavka, ili Context — kontekst), opis, vezuje se za konkretan cilj i opcijski za strategiju, a poseduje attribute isActive (aktiva/neaktivna) i version za praćenje istorijata. Sistem mora podržati deaktivaciju premisa, ali ne i brisanje, čime se čuva istorijat. Sve premise jednog cilja ili strategije moraju biti dostupne za pregled.

3.6. GQM ciljevi, pitanja, ciljne vrednosti i merenja

Za svaki strateški cilj mogu biti definisani jedan ili više GQM ciljeva koji konkretizuju šta treba meriti. Svaki GQM cilj može imati jedno ili više pitanja koja opisuju na koji način se prati ostvarenje. Svako pitanje može imati jednu ili više ciljnih vrednosti (Target) koje definišu naziv, opis i jedinicu merenja. Sistem podržava širok spektar jedinica merenja uključujući procenat, odnose, indekse, vremenske jedinice, monetarne jedinice, metrike kvaliteta softvera i mnoge druge.

Za svaku ciljnu vrednost mogu biti uneta merenja (Measurement) koja sadrže vrednost i tačan datum i vreme merenja. Sistem mora obezbediti konzistentnost podataka i validirani opseg vrednosti pri unosu merenja.

3.7. Procene verovatnoće ostvarenja

Sistem mora podržati kreiranje i upravljanje procenama verovatnoće ostvarenja strateških ciljeva (GoalProbabilityAssessment). Procena se vezuje za konkretan cilj i sadrži verovatnoću (numerička vrednost), napomene i metodu procene. Svaki cilj može imati samo jednu aktivnu

procenu u datom trenutku. Sistem mora validirati da vrednost verovatnoće bude u dozvoljenom opsegu.

3.8. Orkestracija distribuiranih transakcija

Sistem mora podržati izvršavanje složenih operacija koje zahvataju više servisa kroz implementaciju Saga obrasca. Orchestration servis kreira i upravlja SagaWorkflow entitetima, koji se sastoje od niza koraka (SagaStep). Svaki korak ima definisan status (Pending, Completed, Failed, Compensated). U slučaju greške u bilo kom koraku, servis mora automatski pokrenuti kompenzacione akcije za sve prethodno uspešno izvršene korake čime se garantuje konzistentnost sistema.

3.9. Audit log

Sistem mora beležiti sve operacije izmene podataka u centralizovanom audit log-u. Svaki zapis mora sadržati identifikator korisnika koji je izvršio operaciju, naziv entiteta i akcije, vremenski pečat i korelacioni ID koji omogućava praćenje čitavog lanca zahteva kroz sistem. Audit log mora biti dostupan za pregled uz mogućnost filtriranja po relevantnim kriterijumima.

4. Nefunkcionalni zahtevi

4.1. Bezbednost

Sva komunikacija između krajnjih korisnika i sistema odvija se kroz API Gateway koji služi kao jedina ulazna tačka. Korisnička autentifikacija je zasnovana na JWT tokenima sa kratkotrajnim access tokenima (15 minuta) i rotacijom refresh tokena, čime se smanjuje rizik od zloupotrebe ukradenih tokena. Sve interne komunikacije između servisa zaštićene su HMAC potpisom sa vremenskim prozorom od 5 minuta, koji sprečava napade ponovnom upotrebom (replay attacks). Tajni HMAC ključ se konfiguriše kroz environment varijable i nikada nije hardkodovan u izvornom kodu.

4.2. Skalabilnost

Mikroservisna arhitektura sistema direktno podržava horizontalno skaliranje. Svaki servis može biti skaliran nezavisno u skladu sa opterećenjem. Princip odvojenih baza podataka po servisu eliminišu zajedničke tačke zagušenja. Nginx kao API Gateway može biti proširivan upstream blokovima za više instanci svakog servisa.

4.3. Dostupnost i otpornost na greške

Svaki servis izlaže /health endpoint koji vraća trenutni status servisa. Saga obrazac u Orchestration servisu omogućava povrat u konzistentno stanje u slučaju delimičnog neuspeha distribuirane operacije. Separacija servisa znači da kvar jednog servisa ne uzrokuje neposredno kvar čitavog sistema.

4.4. Održivost i razvojni standardi

Svi servisi primenjuju jednobrazne konvencije imenovanja, uniformnu slojevitú arhitekturu i iste obrasce za konfiguraciju dependency injection-a. Zajednički kod je ekstrahovan u dijeljene biblioteke. Svaka izmena koda prolazi kroz GitHub Actions CI pipeline koji automatski gradi i testira sve zahvaćene servise, čime se sprečava integracija neispravnog koda u glavne grane.

4.5. Praćenje i dijagnostika

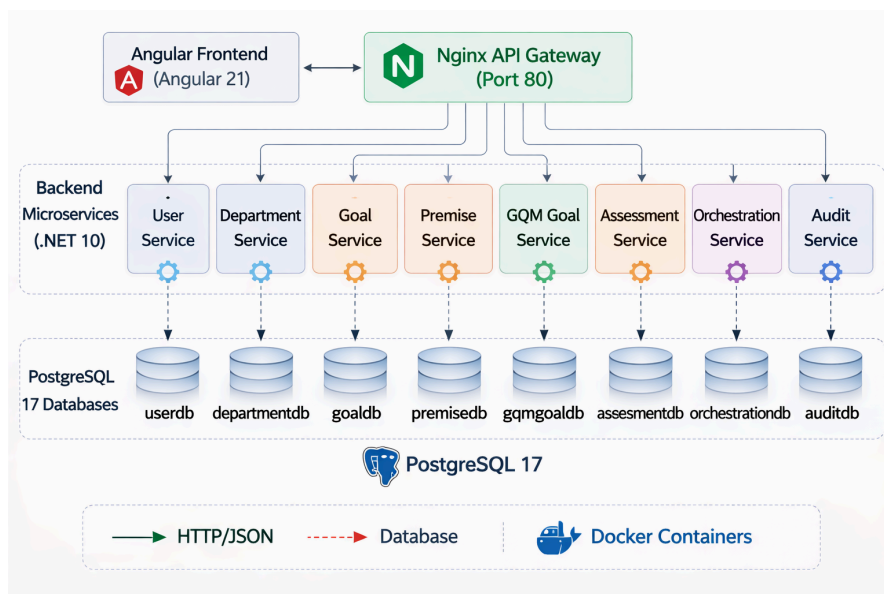
Korelacioni ID jedinstven za svaki zahtev prosleđuje se kroz ceo lanac servisa i zapisuje u audit log, što omogućava praćenje i dijagnostiku problema u distribuiranom okruženju. Standardizovani format grešaka kroz GlobalExceptionHandlerMiddleware osigurava konzistentan odgovor klijentima bez izlaganja internih tehničkih detalja.

5. Arhitektura sistema

5.1. Opšti pregled arhitekture

Sistem GqmPlus projektovan je prema mikroservisnoj arhitekturi u kojoj svaki zasebni funkcionalni domen sistema predstavlja jedan samostalan servis. Ukupno postoji 17 Docker kontejnera koji čine sistem u potpunosti: osam backend servisa, osam izoliranih PostgreSQL baza podataka i jedan Nginx API Gateway kontejner (Slika 3).

Svi backend servisi implementirani su kao ASP.NET Core Web API aplikacije pisane u .NET 10. Baze podataka koriste PostgreSQL 17. Komunikacija između korisnika i sistema odvija se isključivo kroz Nginx koji sluša na portu 80 i rutira zahteve ka odgovarajućim servisima unutar Docker mreže.



Slika 3: Arhitekturni dijagram celog sistema

5.2. Mikroservisni principi

Svaki servis je projektovan prema principu jedinstvene odgovornosti — zadužen je isključivo za upravljanje jednim poslovnim domenom. Servisi su slabo spregnuti (loosely coupled): ne dele kod direktno osim kroz dijeljene biblioteke i ne pristupaju bazama podataka jedni drugh. Svaki servis je nezavisno rasporediv — može se izgraditi, testirati i pokrenuti bez uticaja na ostale servise. Interne komunikacije između servisa koriste isključivo HTTP protokol, bez razmene poruka kroz message broker sistem (direktni sinhroni HTTP pozivi).

5.3. Database-per-service pattern

Sistem primenjuje obrazac odvojene baze podataka po servisu. Ovaj obrazac znači da nijedan servis ne pristupa bazi podataka drugog servisa — svaki servis ima svoju izoliranu PostgreSQL bazu. Svaka baza pokrenuta je unutar sopstvenog Docker kontejnera, a podaci su trajno sačuvani kroz Docker volume-ove. Posledica ovog pristupa je da se sinhronizacija podataka između servisa, tamo gde je potrebna, mora ostvariti kroz eksplicitne HTTP pozive, a ne kroz zajedničke SQL upite.

5.4. Interna slojevita arhitektura servisa

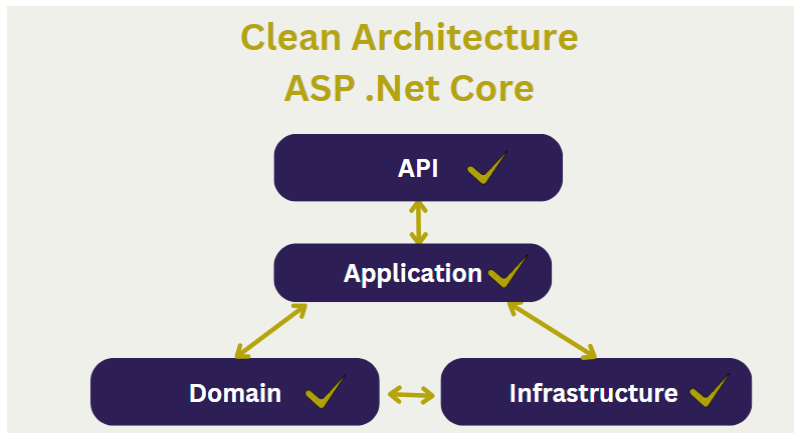
Svaki od osam backend servisa interno prati čistu slojevitú arhitekturu podeljenu u četiri projekta: API, Application, Domain i Infrastructure, uz zasebni Tests projekat. Ovakva organizacija osigurava jasne i stroge granice odgovornosti i jednostavnost testiranja (*Slika 4*).

API sloj ({ServisName}.API) sadrži HTTP kontrolere, middleware komponente i entry point aplikacije (Program.cs). Kontroleri primaju HTTP zahteve, pozivaju odgovarajuće servise iz Application sloja i vraćaju HTTP odgovore. Kontroleri ne sadrže poslovnu logiku.

Application sloj ({ServisName}.Application) nosi poslovnu logiku sistema. Sadrži konkretne implementacije servisa, interfejse servisa i DbContext-a, DTO (Data Transfer Object) klase za ulazne i izlazne podatke, FluentValidation validatore i mapiranja između entiteta i DTO-ova.

Domenski sloj ({ServisName}.Domain) sadrži srž poslovnog modela: entitete, enumeracije, domenski specifične izuzetke i konstante. Ovaj sloj nema zavisnosti prema ostalim slojevima i može biti testiran u potpunoj izolaciji.

Infrastrukturni sloj ({ServisName}.Infrastructure) sadrži implementacione detalje: Entity Framework Core DbContext, konfiguracije entiteta putem Fluent API, HTTP klijente za komunikaciju sa drugim servisima i seed podatke za razvojno okruženje.



Slika 4: Dijagram slojevite arhitekture

6. Opis mikroservisa

6.1. User Service

User Service je centralni servis zadužen za upravljanje korisnicima, ulogama, permisijama i autentifikacijom. Ovaj servis je jedini u sistemu koji izdaje JWT tokene i koji komunicira direktno sa korisnikom u procesu prijavljivanja.

Domenski sloj servisa sadrži sledeće entitete:

- **User** (korisnik sa atributima firstName, lastName, email, passwordHash, isActive),
- **Role** (uloga sa nazivom i opisom),
- **Permission** (permisija sa tekstualnim imenom),
- **RolePermission** (veza many-to-many između uloge i permisija) i
- **UserOrganizationRole** (veza korisnika, organizacije i uloge).

Konstantne vrednosti permisija i uloga definisane su u `UserService.Domain.Constants.Permissions` i `Roles`.

Servis izlaže kontrolere za sledeće putanje:

- **AuthController** na `/api/v1/auth` (login, logout, refresh, me endpoint),
- **UserController** na `/api/v1/user`,
- **RoleController** na `/api/v1/role`,
- **PermissionController** na `/api/v1/permission` i
- **UserOrganizationRoleController** na `/api/v1/userorganizationrole`.

Specifičnost User Servisa u odnosu na ostale servise je primena EF Core migracija automatski pri svakom pokretanju aplikacije — `MigrateAsync()` se poziva u `Program.cs` pre nego što

aplikacija počne da prima zahteve. Data Seeder (u razvojnem okruženju) popunjava bazu inicijalnim korisničkim podacima, ulogama i permisijama. User Service koristi AutoMapper za mapiranja između entiteta i DTO-ova, a FluentValidation za validaciju ulaznih podataka.

Na infrastrukturnom nivou, User Service poseduje HTTP klijent ka Audit Service-u (IAuditClient) za beleženje relevantnih operacija. JWT konfiguracija (SecretKey, Issuer = user-service, Audience = gqmplus-app, ExpiryMinutes = 15) se čita iz environment varijabli i prosleđuje kroz Shared.Auth biblioteku.

6.2. Department Service

Department Service upravljač je organizacionom strukturom sistema. Sadrži dva korena entiteta:

- **Organization** (organizacija sa koordinatama name i description) i
- **Department** (departman koji pripada jednoj organizaciji, sa atributima name, description, organizationId i opcionim managerId).

Servis izlaže dva kontrolera:

- **OrganizationController** na /api/v1/organization i
- **DepartmentController** na /api/v1/department.

Oba kontrolera podržavaju standardne CRUD operacije. Domenski izuzeci NotFoundException i BadRequestException koriste se za signalizaciju grešaka iz poslovnog sloja prema API sloju.

Department Service komunicira jedino sa Nginx API Gatewayjem na ulaznom putu i ne inicira međuservisne HTTP pozive, što ga čini jednim od jednostavnijih servisa u sistemu.

6.3. Goal Service

Goal Service implementira centralnu poslovnu logiku sistema — upravljanje strateškim ciljevima, strategijama i uticajima. Ovaj servis je i komunikacijski najaktivniji: inicira HTTP pozive ka Premise Serviceu, Assessment Serviceu, GQM Goal Serviceu, Orchestration Serviceu i Audit Serviceu.

Domenski entiteti su:

- **Goal** (strateški cilj sa atributima focus, object, magnitude, constraints, status, activeFrom, activeTo, baselineProbability, departmentId),
- **Strategy** (strategija sa name, description, refinementType, goalId, opcionim originStrategyId) i
- **GoalInfluence** (uticaj koji vezuje cilj i strategiju uz description).

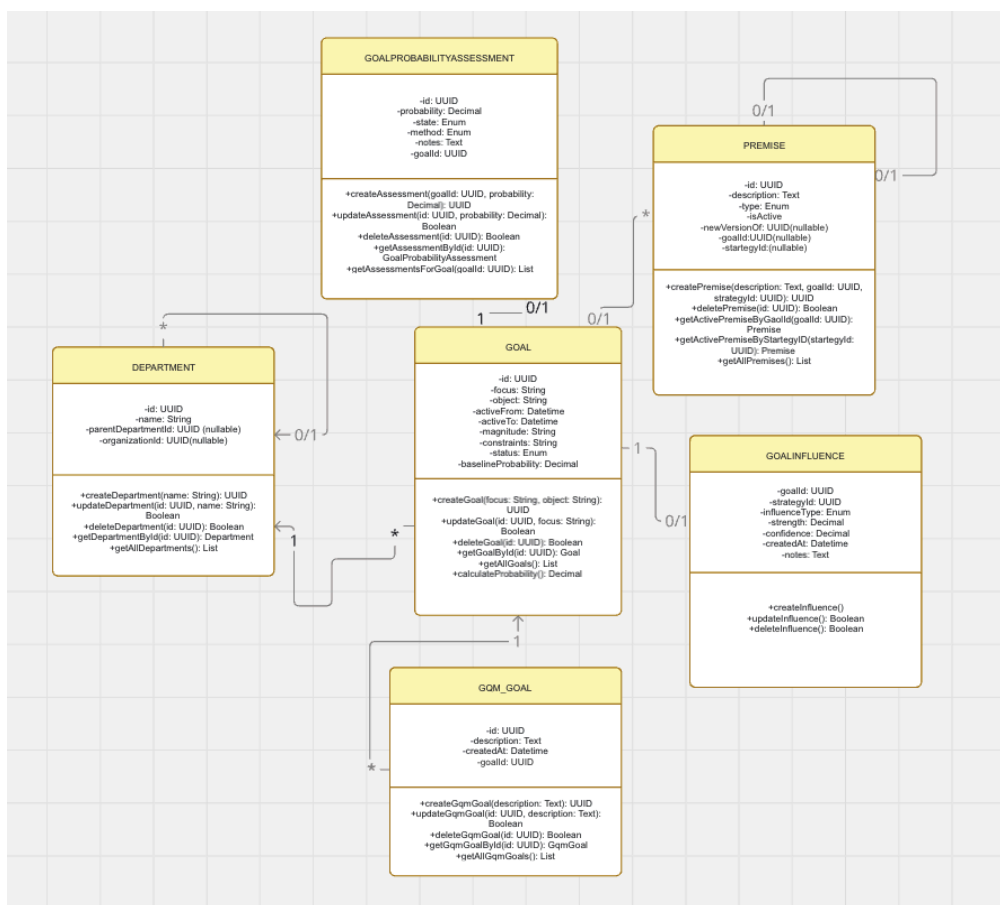
Enumeracije u domenskom sloju:

GoalStatus (Active, OnHold, Completed, Cancelled) označava životni ciklus cilja;
RefinementType (AND, OR) određuje logiku razlaganja strategija;
EffectivenessLevel i **InfluenceType** opisuju prirodu uticaja.

Domenski izuzeci specifični za ovaj servis su *GoalNotFoundException*,
StrategyNotFoundException, *GoalActivationException*, *GoalHierarchyCycleException* i
InvalidGoalStateException. Svaki od ovih izuzetaka hvata *GlobalExceptionHandlerMiddleware* i
pretvara ga u standardizovani HTTP odgovor.

Application sloj registruje tri servisne implementacije: *GoalServiceImpl*, *StrategyServiceImpl* i
GoalInfluenceServiceImpl, svaka dostupna kroz odgovarajući interfejs. *FluentValidation*
validatori proveravaju ispravnost ulaznih DTO-ova pre nego što se obrada preda servisima.

U *Program.cs* od *Goal* Servisa registrovano je pet HTTP klijenata za međuservisnu
komunikaciju: *IPremiseClient* → *PremiseClient*, *IAssessmentClient* → *AssessmentClient*,
IQgmGoalClient → *QgmGoalClient*, *IOrchestrationClient* → *OrchestrationClient* i *IAuditClient* →
AuditClient. Svaki klijent konfigurisan je sa baznom adresom koja se čita iz *environment* varijabli
i dekorisana je sa *HmacDelegatingHandler* i *CorrelationIdDelegatingHandler* za bezbednost i
praćenje zahteva (*Slika 5*).



Slika 5: Dijagram zavisnosti Goal Service-a

6.4. Premise Service

Premise Service upravlja premisama — pretpostavkama i kontekstualnim faktorima koji definišu uslove pod kojima strateški ciljevi važe. Centralni entitet je

Premise sa atributima: `description`, `type`, `goalId`, `opcion`, `strategyId`, `isActive`, `version` i `createdAt`.

Enumeracija:

PremiseType definiše dve vrste premisa:

- **Assumption** (pretpostavka — nešto što se uzima kao tačno ali nije dokazano) i
- **Context** (kontekst — pozadinska informacija koja utiče na cilj).

Verzionašenje premisa (`version`) omogućava praćenje evolucije pretpostavki tokom vremena.

Domenski izuzeci servisa precizno opisuju moguće greške: *PremiseNotFoundException* se koristi kada tražena premissa ne postoji, *PremiseAlreadyDeactivatedException* kada se pokuša ponovna deaktivacija već neaktivne premise, *PremisesNotFoundByGoalException* i *PremisesNotFoundByStrategyException* za slučajeve nedostajućih premisa u kontekstu konkretnog cilja ili strategije.

Premise Service komunicira sa Orchestration Servisom (za koordinaciju operacija kroz Saga obrazac) i sa Audit Servisom (za beleženje izmena). Servis prima HMAC zaštićene zahteve od Goal Servisa koji na taj način dohvata premise vezane za ciljeve i strategije.

6.5. GQM Goal Service

GQM Goal Service implementira drugu hijerarhijsku dimenziju sistema — GQM (Goal-Question-Metric) strukturu koja razlaže strateške ciljeve na merljive komponente. Ovaj servis je organizovan oko četiri entiteta koji formiraju agragatnu hijerarhiju merenja.

- **GqmGoal** entitet vezuje GQM cilj za strateški cilj (kroz `goalId`) i sadrži opis koji precizira šta se meri.
- **Question** entitet predstavlja pitanje koje opisuje jedan aspekt ostvarenja GQM cilja, vezano za njega kroz `gqmGoalId`.
- **Target** (ciljna vrednost) definiše konkretni merljivi cilj vezan za pitanje, sa atributima `name`, `description`, `unit` i `questionId`.
- **Measurement** (merenje) predstavlja konkretno izmerenu vrednost, sa atributima `value`, `measuredAt` i `targetId`.

Enumeracija:

Unit - u domenskom sloju sadrži izuzetno bogat skup jedinica merenja koji pokriva sve potrebe poslovnog domena:

- *opšte numeričke jedinice* (Percentage, Ratio, Index, Score, Count),

- *vremenske jedinice* (Milliseconds, Seconds, Minutes, Hours, Days, Weeks, Months, Years),
- *monetarne jedinice* (Currency, CurrencyPerHour, BudgetVariance, RevenuePerUnit),
- *softverske metrike* (LinesOfCode, CodeCoveragePercentage, BuildDurationMinutes, DeploymentFrequency, CycleTimeDays),
- *jedinice opterećenja* (ThroughputPerSecond, ResponseTimeMilliseconds, LatencyMilliseconds),
- *kadrovske metrike* (Employees, SatisfactionScore, AttritionRate) i
- *mnoge druge*, uključujući Custom i Other za nestandardne slučajeve.

Application sloj koristi *ApplicationServiceExtensions* klasu za registraciju DI servisa, konzistentno sa ostatkom arhitekture. Kontroleri su organizovani po entitetima: GqmGoalController, QuestionController, TargetController i MeasurementController. Dokumentovani domenski izuzeci (NotFoundException, ConflictException, ValidationException) osiguravaju jasnu propagaciju grešaka.

6.6. Assessment Service

Assessment Service zadužen je za upravljanje procenama verovatnoća ostvarenja strateških ciljeva. Centralni entitet **GoalProbabilityAssessment** čuva informacije o procenjenoj verovatnoći, napomenama, metodi procene i stanju procene.

Enumeracija:

AssessmentMethod definiše moguće metode kojima je procena izvršena, dok **AssessmentState** prati životni ciklus procene.

Domenski izuzeci precizno definišu sve moguće greške: *AssessmentNotFoundException* za nepostojeću procenu, *AssessmentAlreadyExistsException* kada se pokuša kreiranje duplikata, *AssessmentByGoalNotFoundException* kada za dati cilj ne postoji procena i *InvalidProbabilityException* kada unesena vrednost verovatnoće nije u dozvoljenom opsegu.

Assessment Service komunicira sa tri servisa: GoalClient ka Goal Service-u za validaciju da procenjivani cilj postoji, OrchestrationClient ka Orchestration Service-u za koordinaciju složenijih operacija i AuditClient ka Audit Service-u za reviziju operacija. Koristi FluentValidation za validaciju ulaznih podataka kroz CreateAssessmentValidator i UpdateAssessmentValidator.

6.7. Orchestration Service

Orchestration Service implementira Saga obrazac (*Slika 6*) za orkestraciju distribuiranih transakcija kroz sistem. Ovaj servis je arhitekturno specifičan jer mu ne pristupaju krajnji korisnici direktno — ne koristi JWT autentifikaciju, već isključivo **HMAC** zaštitu za međuservisnu komunikaciju.

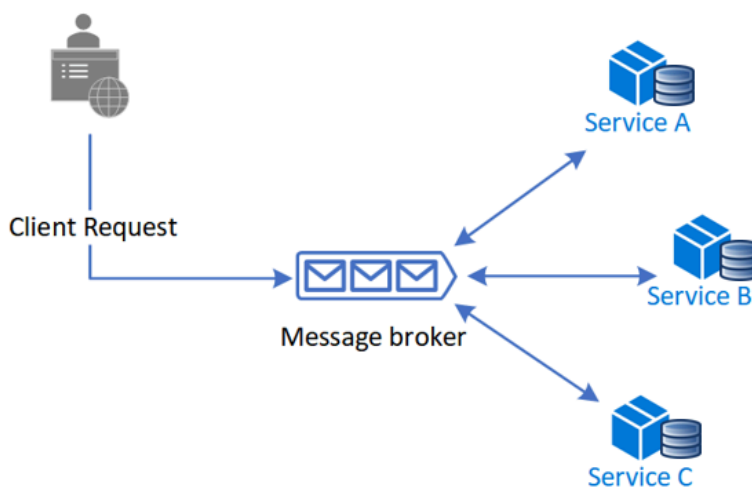
Domenski model servisa obuhvata dva entiteta:

- **SagaWorkflow** koji predstavlja jednu distribuiranu transakciju kao celinu i
- **SagaStep** koji predstavlja jedan korak unutar te transakcije.

SagaStatus enumeracija definiše stanja toka rada (Pending, InProgress, Completed, Failed, Compensated), dok **SagaStepStatus** prati status svakog pojedinačnog koraka (Pending, Completed, Failed, Compensated). **WorkflowStep** enumeracija definiše sve moguće tipove koraka u sistemu.

Logika servisa funkcioniše na sledeći način: kada servis koji inicira kompleksnu operaciju (npr. Goal Service) zatraži od Orchestration Service-a pokretanje Sage, servis kreira SagaWorkflow i odgovarajuće SagaStep zapise. Svaki korak se izvršava redom — uspešan korak prelazi u status Completed, dok neuspešan prelazi u Failed. U slučaju neuspeha, Orchestration Service pokreće kompenzacione HTTP zahteve ka prethodno uspešnim koracima, pozivajući endpoint-e koji poništavaju efekte tih operacija. CompensationHttpClient koristi apsolutne URL-ove za kompenzacione zahteve jer kompenzacija može biti upućena ka bilo kom servisu u sistemu.

Domenski izuzeci servisa su *SagaNotFoundException*, *SagaAlreadyExistsException* i *SagaAlreadyCompensatedException*. EF Core migracije se automatski primenjuju pri pokretanju servisa.



Slika 6: Dijagram Saga obrasca

6.8. Audit Service

Audit Service odgovoran je za centralizovano logovanje svih operacija izmene podataka u sistemu. Kao i Orchestration Service, ne pruža direktan pristup krajnjim korisnicima i ne koristi JWT autentifikaciju. Komunikacija ka Audit Service-u isključivo je HMAC zaštićena.

Centralni entitet je **AuditLog** koji čuva zapis o svakoj akciji u sistemu. U aplikacionom sloju, AuditLogService prima zahteve za kreiranje novih zapisa i obezbeđuje pregled istorijata uz filtriranje po kriterijumima definisanim u AuditLogFilter DTO-u.

Ulazni zahtev `CreateAuditLogRequest` validira se kroz `CreateAuditLogRequestValidator`. `AuditLogResponse` je izlazni DTO koji se vraća klijentima (uglavnom administratorima koji pregledaju revizorski trag). Domenski izuzetak `AuditLogNotFoundException` koristi se u slučaju nepostojećeg zapisa.

7. Dijeljene biblioteke

7.1. Shared.HMAC

Biblioteka `Shared.HMAC` implementira bezbednosni mehanizam za autentifikaciju međuservisnih HTTP poziva zasnovanu na HMAC (Hash-based Message Authentication Code) algoritmu.

`HmacService` klasa implementira računanje HMAC potpisa. Potpis se računa od konkatencije tela zahteva, vremenskog pečata (Unix timestamp) i tajnog ključa, primenom SHA-256 hash algoritma. Ovaj pristup osigurava i autentičnost (poznavalac tajnog ključa) i integritet (bilo koja izmena tela zahteva invalidira potpis). Vremenski pečat u potpisu sprečava napade ponovnom upotrebom starih zahteva — middleware validira da vremenski pečat nije stariji od 5 minuta.

`HmacDelegatingHandler` je ASP.NET Core delegating handler koji se dodaje u HTTP klijent lanac. Automatski dodaje dva zaglavlja na svaki odlazni zahtev: `X-HMAC-Signature` (izračunati HMAC potpis) i `X-HMAC-Timestamp` (trenutni vremenski pečat).

`HmacMiddleware` je ASP.NET Core middleware koji se izvršava na ulaznoj strani servisa. Čita `X-HMAC-Signature` i `X-HMAC-Timestamp` zaglavlja, rekonstruiše očekivani potpis na osnovu tela zahteva, primljenog pečata i tajnog ključa, i upoređuje ga sa primljenim potpisom. Ako validacija ne prođe ili je pečat stariji od 5 minuta, middleware vraća HTTP 401 odgovor. Validacija se preskače za putanje definisane kao whitelist: `/health`, `/swagger` i `/weatherforecast`.

`ServiceCollectionExtensions` pruža `AddHmacAuthentication(secretKey)` extension metodu koja registruje sve HMAC komponente u DI kontejner.

7.2. Shared.Auth

Biblioteka `Shared.Auth` implementira zajedničku JWT autentifikacijsku infrastrukturu koja se koristi u svim servisima koji komuniciraju direktno sa krajnjim korisnicima.

`JwtAuthExtensions` klasa pruža `AddJwtAuthentication(configuration)` extension metodu koja konfiguriše ASP.NET Core autentifikacioni middleware sa JWT Bearer šemom. Čita parametre iz konfiguracije: `Jwt__SecretKey` (tajni ključ za potpisivanje tokena), `Jwt__Issuer` (izdavalac — user-service), `Jwt__Audience` (publika — gqmplus-app). `JwtSettings` je POCO klasa koja modelira ovu konfiguraciju.

ClaimsPrincipalExtensions sadži extension metode nad ClaimsPrincipal koje omogućavaju jednostavno čitanje korisničkih podataka iz JWT tokena, kao što su identifikator korisnika i lista permisija.

OrganizationContextMiddleware čita X-Organization-Id zaglavlje iz HTTP zahteva i smešta ga u HttpContext.Items, čime je organizacioni kontekst dostupan svim komponentama pipeline-a tokom obrade jednog zahteva. Ova vrednost postavlja se od strane Nginx API Gateway-a koji je čita iz korisničkog zahteva.

RequirePermissionAttribute je custom action atribut koji se može primeniti na kontrolere ili metode i proverava da li autentifikovani korisnik ima potrebnu permisiju pre nego što dozvoli pristup endpoint-u.

7.3. Shared.Contracts

Biblioteka Shared.Contracts definiše zajedničke DTO-ove za standardizovanu komunikaciju, pre svega za paginaciju.

PaginationRequest je ulazni DTO sa atributima PageNumber, PageSize i OrderBy, koji se koristi kao parametar upita (query string) za sve endpoint-e koji vraćaju liste entiteta.

PaginationResponse<T> je generički izlazni DTO koji sadrži: Items (lista stavki tekuće stranice), TotalCount (ukupan broj stavki), PageNumber, PageSize i TotalPages. Frontendski PaginatedResponse<T> interfejs u *api.models.ts* usklađen je sa ovim DTO-om.

7.4. Shared.ErrorHandling

Biblioteka Shared.ErrorHandling implementira dva ortogonalna mehanizma: standardizovano upravljanje greškama i propagaciju korelacionog ID-a.

GlobalExceptionHandlerMiddleware je middleware koji hvata sve neobrađene izuzetke u pipeline-u pre nego što se oni propagiraju ka klijentu. Beleži grešku u log i vraća StandardErrorResponse u formatu koji je ujednačen kroz sve servise. StandardErrorResponse sadrži HTTP statusni kod, poruku greške i detalje, bez izlaganja internih informacija o steku poziva.

ErrorHandlingExtensions pruža UseStandardizedExceptionHandler() extension metodu koja registruje GlobalExceptionHandlerMiddleware u pipeline. Svaki servis poziva ovu metodu na samom početku middleware pipeline-a, odmah nakon korelacionog ID-a, čime se osigurava da nijedna greška ne "pobegne" nestrukturisana ka klijentu.

CorrelationIdMiddleware čita X-Correlation-Id zaglavlje iz dolaznog zahteva i smešta vrednost u HttpContext.Items gde je dostupna tokom čitavog životnog ciklusa zahteva. Ova vrednost se upisuje u svaki audit log zapis, čime se postiže praćenje zahteva kroz ceo sistem.

CorrelationIdDelegatingHandler je delegating handler koji se dodaje u lanac HTTP klijenata za međuservisne pozive. Čita korelacioni ID iz trenutnog konteksta i dodaje ga kao X-Correlation-Id zaglavlje na svaki odlazni zahtev, čime se ID propagira duž celog lanca poziva.

8. Opis baze podataka

8.1. Database-per-service pristup

Svaki od osam mikroservisa poseduje svoju izoliranu PostgreSQL 17 bazu (*Tabela 1*) podataka koja se pokreće unutar sopstvenog Docker kontejnera i čiji podaci su trajno sačuvani kroz Docker named volume. Tabela u nastavku pregledno prikazuje mapiranje servisa ka bazama:

Servis	Baza podataka	DB kontejner	Eksterni port
user-service	userdb	user-db	5435
department-service	departmentdb	department-db	5440
goal-service	goaldb	goal-db	5433
premise-service	premisedb	premise-db	5432
gqm-goal-service	gqmgoaldb	gqm-goal-db	5436
assessment-service	assessmentdb	assessment-db	5434
orchestration-service	orchestrationdb	orchestration-db	5437
audit-service	auditdb	audit-db	5438

Tabela 1: Database-per-service pristup

Svaka baza radi sa korisničkim nalogom postgres, lozinkom postgres i datim imenom baze, što je konfigurisano kroz environment varijable POSTGRES_DB, POSTGRES_USER i POSTGRES_PASSWORD unutar Docker Compose fajla.

8.2. Entity Framework Core konfiguracija

Svi servisi koriste **Entity Framework Core** sa **Npgsql PostgreSQL** provajderom (UseNpgsql). DbContext svake aplikacije implementira interfejs koji je definisan u Application sloju (npr. IGoalDbContext, IOrchestrationDbContext), a konkretna implementacija se nalazi u Infrastructure sloju. Ovakav pristup osigurava inverziju zavisnosti i omogućava testiranje bez potrebe za stvarnom bazom podataka.

Konfiguracija entiteta vrši se isključivo putem Fluent API u Infrastructure/Configurations direktorijumu, koristeći zasebne konfiguracione klase (npr. GoalProbabilityAssessmentConfiguration). Format connection stringa koji se koristi je

Host={service}-db;Port=5432;Database={service}db;Username=postgres;Password=postgres, a servis ga čita iz environment varijable DATABASE_URL.

8.3. Migracije i seed podaci

Strategija migracija razlikuje se između servisa zavisno od složenosti modela. User Service i Orchestration Service automatski primenjuju EF Core migracije pri svakom pokretanju korišćenjem `await dbContext.Database.MigrateAsync()` u `Program.cs`. Ovo garantuje da je shema baze podataka uvek sinhronizovana sa domenskim modelom.

Seed podaci su namenjeni isključivo razvojnom okruženju i omogućavaju nezavisno testiranje servisa bez potrebe za ručnim unosom podataka. Goal Service poziva `GoalDbSeeder.SeedAsync()` unutar `if (app.Environment.IsDevelopment())` bloka. User Service poziva `DataSeeder.SeedAsync()` koji popunjava inicijalnu listu korisnika, uloga i permisija. Svi seed podaci moraju biti uklonjeni ili prilagođeni pre produkcijskog puštanja u rad.

8.4. Volumi za trajnost podataka

Svaka baza podataka konfigurisana je sa imenovanim Docker volume-om koji osigurava trajnost podataka između pokretanja: `user-db-data`, `department-db-data`, `goal-db-data`, `premise-db-data`, `gqm-goal-db-data`, `assessment-db-data`, `orchestration-db-data` i `audit-db-data`. Brisanje volume-ova komandom `docker compose down -v` resetuje podatke u bazama, što je korisno u razvojnom okruženju za vraćanje u početno stanje.

9. Komunikacija između servisa

9.1. HTTP klijenti i `IHttpClientFactory` obrazac

Sva međuservisna komunikacija odvija se isključivo putem sinhronih HTTP poziva. Svaki servis koji komunicira sa drugim servisom koristi .NET-ov `IHttpClientFactory` mehanizam koji se konfiguriše kroz `AddHttpClient<TInterface, TImplementation>()` u `Program.cs`. Typed HTTP klijenti obezbeđuju da svaki klijent ima jasno definisan interfejs, baznu adresu target servisa i pipe-line sa potrebnim delegating handler-ima. Bazne adrese se čitaju iz environment varijabli (npr. `Services__PremiseService`, `Services__AuditService`) uz podrazumevane vrednosti koje odgovaraju Docker imenima kontejnera.

Mapa svih međuservisnih komunikacionih veza prikazana je u nastavku. Goal Service komunicira sa pet servisa: Premise Service (dohvatanje premisa za cilj), Assessment Service (dohvatanje procena za cilj), GQM Goal Service (dohvatanje GQM ciljeva), Orchestration Service (pokretanje Saga tokova) i Audit Service (beleženje izmena). Premise Service komunicira sa Orchestration Servisom i Audit Servisom. GQM Goal Service komunicira sa Orchestration Servisom i Audit Servisom. Assessment Service komunicira sa Goal Servisom (validacija postojanja cilja), Orchestration Servisom i Audit Servisom. Orchestration Service

komunicira sa Audit Servisom i sa kompenzacionim endpoint-ima servisa putem apsolutnih URL-ova. User Service komunicira sa Audit Servisom.

9.2. HMAC autentifikacija međuservisnih poziva

Svaki HTTP klijent registrovan za međuservisnu komunikaciju dekorisan je sa HmacDelegatingHandler-om iz biblioteke Shared.HMAC. Ovaj handler automatski dodaje X-HMAC-Signature i X-HMAC-Timestamp zaglavlja na svaki odlazni zahtev. Na strani prijema, HmacMiddleware validira ova zaglavlja pre nego što zahtev dospe do kontrolera.

Tajni ključ koji se koristi za računanje potpisa jednak je na svim servisima jer se svi oslanjaju na istu environment varijablu HMAC_SECRET_KEY definisanu u Docker Compose konfiguraciji. U produkcijskom okruženju ovaj ključ mora biti zamenjen jakim, nasumično generisanim vrednostima.

9.3. Korelacioni ID i praćenje zahteva

Korelacioni ID (X-Correlation-Id) jedinstven je identifikator koji se generiše na nivou Nginx API Gateway-a za svaki ulazni zahtev. Nginx koristi map direktivu koja proverava da li korisnik (klijentska strana) šalje sopstveni ID — ako šalje, koristi se taj ID, a ako ne šalje, Nginx generiše jedinstveni ID koristeći svoju internu promenljivu \$request_id. Nginx dodaje ovaj ID u odgovarajuće zaglavlje svakog zahteva koji prosleđuje servisu.

Na strani servisa, CorrelationIdMiddleware čita primljeni ID i smešta ga u HttpContext.Items. Svaki međuservisni odlazni poziv prolazi kroz CorrelationIdDelegatingHandler koji preuzima ID iz konteksta i dodaje ga kao zaglavlje novog zahteva. Na taj način, isti korelacioni ID prisutan je u svim zahtevima koji su nastali iz jednog originalnog korisničkog zahteva, a upisuje se i u svaki AuditLog zapis. Ovo omogućava da se kroz Audit Service rekonstruiše kompletan tok jedne operacije kroz čitav sistem.

9.4. Saga obrazac i kompenzacione transakcije

Orchestration Service implementira Saga obrazac za koordinaciju operacija koje zahvataju više servisa. Kada jedna operacija zahteva koordinisane izmene u više servisa, pozivajući servis najpre obaveštava Orchestration Service o pokretanju novog toka rada (SagaWorkflow). Orchestration Service kreira workflow instancu i beleži svaki korak (SagaStep) sa odgovarajućim statusom.

Ukoliko u nekom koraku dođe do greške, Orchestration Service ne može primeniti standardnu bazu podataka transakciju jer su podaci raspoređeni po različitim bazama. Umesto toga, Orchestration Service pokreće kompenzacione HTTP zahteve ka svim prethodno uspešnim koracima. Svaki pozivajući servis izlaže posebne endpoint-e za kompenzaciju koji poništavaju efekat već izvršene operacije. CompensationHttpClient koristi apsolutne URL-ove jer kompenzacioni endpoint-i mogu biti na različitim servisima u zavisnosti od tipa koraka.

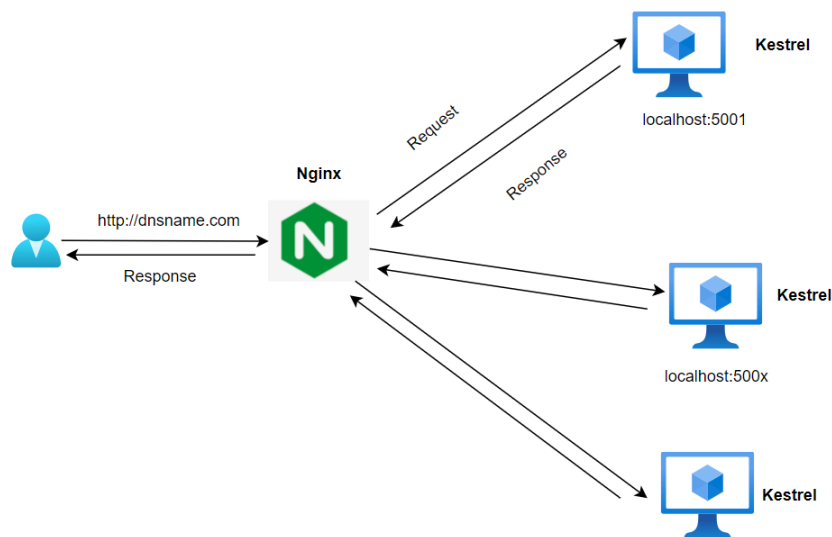
9.5 RabbitMQ

Poslovna logika i asinhrona priroda određenih akcija diktiraju primenu adekvatnog message brokera, pa se tako RabbitMQ koristi unutar usvojene mikroservisne arhitekture. Servisi kao što su Goal Service, Orchestration Service i Audit Service komuniciraju slanjem i preuzimanjem poruka iz reda obezbeđujući takozvanu Event-Driven arhitekturu. Ovakav asinhroni model značajno smanjuje vreme čekanja HTTP poziva unutar mrežnih petlji, a sama obrada se odvija u pozadini (background) garantujući trajnost poruka i otpornost sistema na greške i padove servisa.

10. API Gateway

10.1. Nginx kao reverzni proksi

Nginx Alpine kontejner konfigurisan je kao API Gateway sistema — jedina ulazna tačka kroz koju sav eksterni saobraćaj ulazi u sistem (*Slika 7*). Sluša na portu 80 i komunicira sa backend servisima unutar Docker interne mreže, koristeći DNS imena kontejnera (npr. goal-service:8080, user-service:8080). Svi interni portovi servisa su 8080, a eksterni portovi na kojima su dostupni direktno (izvan Nginx-a) su različiti i koriste se isključivo za direktno testiranje u razvojnom okruženju.



Slika 7: Nginx kao jedina ulazna tačka

10.2. URL rutiranje

Nginx konfiguracija definiše upstream blok za svaki od osam servisa i zatim koristi location blokove za rutiranje zahteva prema URL putanji. Sledeća tabela (*Tabela 2*) prikazuje kompletno mapiranje putanja:

Servis	API putanje
Department Service	/api/v1/department, /api/v1/organization
Goal Service	/api/v1/goal, /api/v1/strategy, /api/v1/goalinfluence
Premise Service	/api/v1/premise
GQM Goal Service	/api/v1/gqmggoal, /api/v1/question, /api/v1/target, /api/v1/measurement
Assessment Service	/api/v1/assessment
User Service	/api/v1/user, /api/v1/auth, /api/v1/permission, /api/v1/role, /api/v1/userorganizationrole
Orchestration Service	/api/v1/workflow
Audit Service	/api/v1/audit

Tabela 2: Rutiranje zahteva prema URL putanji

10.3. Prosleđivanje zaglavlja

Svaki location blok u Nginx konfiguraciji prosleđuje skup standardnih i aplikacijski specifičnih zaglavlja. `proxy_set_header Host $host` postavljam ispravno poreklo zahteva. `X-Real-IP` i `X-Forwarded-For` prenose IP adresu izvornog klijenta. `X-Forwarded-Proto` prenosi protokol. `Authorization` zaglavlje koje nosi JWT token krajnjeg korisnika prosleđuje se svim servisima koji zahtevaju autentifikaciju. `X-Organization-Id` zaglavlje prenosi identifikator organizacije u čijem kontekstu klijent radi. `X-Correlation-Id` zaglavlje prenosi jedinstveni identifikator zahteva.

10.4. Generisanje korelacionog ID-a

Nginx konfiguracija koristi `map` direktivu za dinamičko generisanje korelacionog ID-a: `map $http_x_correlation_id $correlation_id { default $http_x_correlation_id; "" $request_id; }`. Ovo znači da ako klijent pošalje `X-Correlation-Id` zaglavlje, ta vrednost se prihvata i propagira. Ako zaglavlje nije prisutno, Nginx generiše jedinstveni ID putem `$request_id`. Nginx zatim dodaje ovu vrednost u odgovor (`add_header X-Correlation-Id $correlation_id always`) i prosleđuje je ka backend servisima.

11. Frontend aplikacija

11.1. Arhitektura Angular aplikacije

Frontend aplikacija sistema `GqmPlus` implementirana je u Angular 21 i koristi isključivo standalone komponente bez `NgModule`-ova, što je moderni Angular pristup koji pojednostavljuje strukturu aplikacije. Angular Material 21 koristi se za UI komponentnu biblioteku uz Angular CDK. Aplikacija je pisana u TypeScript 5.9, a Prettier je konfigurisan za automatsko formatiranje koda.

Aplikacija je organizovana prema feature-first pristupu unutar src/app direktorijuma. Direktorijum core/ sadrži jezgro aplikacije — servise, modele, guards i interceptore koji su dijeljeni kroz čitavu aplikaciju. Direktorijum features/ sadrži feature-specifične komponente grupisane prema funkcionalnom domenu. Direktorijum layout/ sadrži komponente koje čine vizuelni okvir aplikacije. Direktorijum shared/ sadrži deljene UI komponente za ponovnu upotrebu.

11.2. Rutiranje i lazy loading

Rutiranje aplikacije konfigurisano je u app.routes.ts korištenjem Angular Router-a sa withComponentInputBinding(). Sve feature komponente se učitavaju lenji (lazy loading) putem dinamičkih importa (loadComponent: () => import(...).then(...)), što znači da se kod za svaku stranicu učitava tek kad korisnik navigira ka toj putanji.

Koren putanje (/) preusmjerava na /dashboard. Auth rute (/auth/login, /auth/forbidden) nemaju layout shell i dostupne su bez autentifikacije. Sve ostale rute su upakovane unutar LayoutComponent-a i zaštićene sa authGuard-om koji proverava da li je korisnik prijavljen. Feature rute dalje zahtevaju specifične permisije putem permissionGuard-a.

Konkretno zaštićene rute su: /dashboard (bez posebnih permisija), /goals i subrute (/goals/new, /goals/:id) sa permisijama view_goals i create_goals, /analytics sa permisijom view_analytics, te admin rute /admin/organizations, /admin/departments, /admin/users, /admin/roles sa odgovarajućim admin permisijama.

11.3. Autentifikacija i upravljanje stanjem

AuthService je centralni singleton servis koji upravlja autentifikacionim stanjem aplikacije korištenjem BehaviorSubject<AuthState>. Stanje expose-uje se kao skup observabla: state\$, user\$, isAuthenticated\$ i token\$, koji omogućavaju reaktivne komponente koje se automatski ažuriraju na promene autentifikacionog stanja.

Pri pokretanju aplikacije, APP_INITIALIZER koji je registrovan u app.config.ts proverava da li postoje tokeni u localStorage. Ako postoje, poziva authService.loadUser() koji šalje GET /api/v1/auth/me zahtev i učitava korisnički kontekst pre nego što se aplikacija iscrta na ekranu. Time se osigurava da korisnik koji osvježi stranicu ostaje prijavljen bez ponovnog unosa kredencijala.

Tok prijavljivanja odvija se u tri koraka: POST zahtev ka /api/v1/auth/login sa kredencijalima, čuvanje primljenih tokena u localStorage, i GET zahtev ka /api/v1/auth/me za učitavanje korisničkih podataka. Odjava se izvršava pozivom POST /api/v1/auth/logout i brisanjem tokena iz localStorage, praćenim preusmjeravanjem na login stranicu.

AuthState interfejs čuva:

- user (korisnički objekat sa email-om, imenom, prezimenom, organizationId, listom permissions i listom managedDepartmentIds),
- accessToken,

- refreshToken i
- isAuthenticated flag.

11.4. API servisi i modeli

Svaki poslovni domen ima odgovarajući API servis u core/api/ direktorijumu: GoalApiService, DepartmentApiService, PremiseApiService, GqmApiService, AssessmentApiService i UserApiService. Ovi servisi enkapsuliraju sve HTTP pozive ka backendu i koriste Angular HttpClient.

Svi TypeScript interfejsi koji modeliraju podatke su centralizovani u api.models.ts. Modeli uključuju:

- **Goal,**
- **GoalDetails** (Goal sa ugnjezdenom Strategy, listom Premise, listom Assessment i listom GqmGoal),
- **Strategy,**
- **GoalInfluence,**
- **Department,**
- **Organization,**
- **Premise,**
- **Assessment,**
- **GqmGoal,**
- **Question,**
- **Target,**
- **Measurement,**
- **User i**
- **Role.**

Pored ovih, definisani su odgovarajući *Request interfejsi za operacije kreiranja i izmene.

PaginationParams i PaginatedResponse<T> su usklađeni sa Shared.Contracts bibliotekom na backendu.

11.5. Interceptor

Aplikacija koristi dva HTTP interceptora registrovana u app.config.ts kroz withInterceptors([authInterceptor, errorInterceptor]).

AuthInterceptor presreće svaki odlazni HTTP zahtev i dodaje mu Authorization: Bearer {accessToken} zaglavlje ako je korisnik prijavljen. Pored toga, dodaje X-Organization-Id zaglavlje sa identifikatorom tekuće organizacije iz korisničkog konteksta, što Nginx dalje prosleđuje backend servisima.

ErrorInterceptor globalno obrađuje HTTP greške. Kada backend vrati 401 status, interceptor pokušava da osveži access token pozivom endpointa za refresh. Ako refresh uspe, originalni

zahtev se automatski ponavlja sa novim tokenom. Ako refresh ne uspe, korisnik se preusmjerava na login stranicu. Ostale greške se obrađuju i propagiraju na odgovarajući način.

11.6. Zaštita ruta

AuthGuard je Angular funkcionalni guard koji proverava `authService.currentState.isAuthenticated` pre aktivacije zaštićene rute. Ako korisnik nije autentifikovan, guard preusmjerava na `/auth/login`.

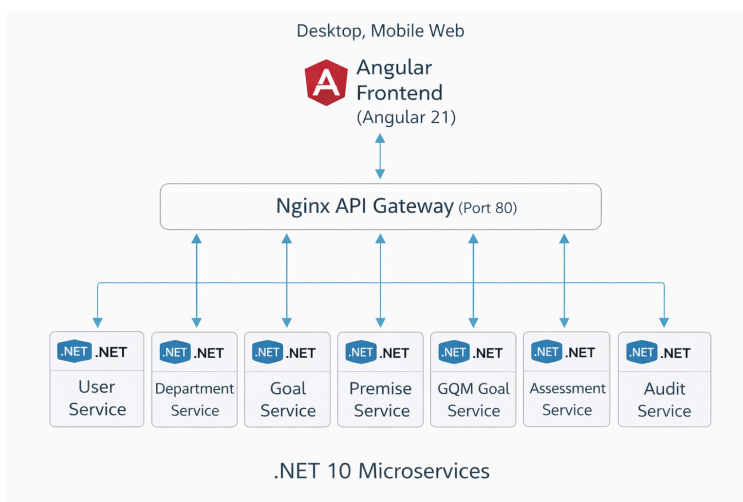
PermissionGuard proverava da li korisnik ima potrebne permisije definisane u data objektu rute. Rute mogu specificirati ili `permissions` (lista permisija od kojih korisnik mora imati sve) ili `anyPermissions` (lista permisija od kojih je dovoljna jedna). Permisije se proveravaju prema listi permisija u korisničkom objektu. Ako korisnik nema dovoljno prava, preusmjerava se na `/auth/forbidden` stranicu.

12. Frontend-backend interakcija

12.1. Tok komunikacije

Angular frontend nikada ne komunicira direktno sa pojedinačnim backend servisima — sva komunikacija teče kroz Nginx API Gateway na portu 80 (Slika 8). U razvojnem okruženju, Angular CLI dev server koristi `proxy.conf.json` koji preusmjerava `/api` zahteve ka Nginx-u, čime se izbegavaju CORS problemi.

Svaki HTTP zahtev koji Angular `HttpClient` pošalje prolazi kroz interceptor lanac koji dodaje `Authentication` i `Organization-Id` zaglavlja. Zahtev stiže do Nginx-a koji mu dodaje korelacioni ID i rutira ga ka odgovarajućem backend servisu na osnovu URL putanje. Backend servis obrađuje zahtev i vraća odgovor koji se vraća do Angular klijenta.



Slika 8: Sekvencijalni dijagram jednog tipičnog zahteva

12.2. Format zahteva i odgovora

Komunikacija između frontenda i backenda koristi JSON format. Zaglavlje Content-Type: application/json se automatski postavlja od strane Angular HttpClient-a za POST i PUT zahteve. Odgovori koji vraćaju liste podataka koriste PaginatedResponse<T> format sa atributima items, totalCount, pageNumber, pageSize i totalPages.

12.3. Rukovanje greškama

Greške nastale na backendu vraćaju se u standardizovanom StandardErrorResponse formatu koji definiše Shared.ErrorHandling biblioteka. Ovaj format sadrži statusni kod i opis greške bez internih tehničkih detalja. ErrorInterceptor na frontendu hvata HTTP greške, razlikuje greške autentifikacije (401) od ostalih i pruža mogućnost prikaza korisničkih poruka o greškama.

13. DevOps postavka

13.1. Docker i kontejnerizacija

Svaki od osam backend servisa pakuje se u Docker sliku korištenjem višefaznog (multi-stage) Dockerfile-a. Ovaj pristup osigurava da finalna produkcijska slika bude što manja jer sadrži samo ASP.NET Core runtime bez .NET SDK alata.

Prva faza (base) polazi od mcr.microsoft.com/dotnet/aspnet:10.0 slike i izlaže port 8080. Druga faza (build) koristi mcr.microsoft.com/dotnet/sdk:10.0 sliku za kompajliranje i pakovanje aplikacije. U ovoj fazi se najpre kopiraju samo .csproj fajlovi deljenih biblioteka i projekata servisa, a zatim se pokreće dotnet restore — ovo koristi Docker slojno keširanje (layer caching) jer se restore pokreće ponovo samo kada se promene .csproj fajlovi, a ne svaki put kada se promeni izvorni kod. Nakon toga kopira se ceo izvorni kod i pokreće dotnet publish -c Release. Treća finalna faza kopira publishovanu aplikaciju iz build faze i postavlja ENTRYPOINT na pokretanje DLL datoteke.

13.2. Docker Compose orkestracija

docker-compose.yaml fajl orkestira 17 kontejnera sistema. Za svaki backend servis konfiguriše se: build.dockerfile putanja ka **Dockerfile**-u, container_name za jasnu identifikaciju, ports mapiranje (interni 8080 na specifični eksterni port), niz environment varijabli koji uključuje okruženje (ASPNETCORE_ENVIRONMENT=Development), konekcioni string baze, HMAC ključ, JWT konfiguraciju i URL-ove zavisnih servisa, te depends_on za bazu podataka servisa.

Svaki PostgreSQL kontejner konfiguriše se sa odgovarajućim imenom baze, korisničkim nalogom, lozinkom i volume-om za trajnost podataka. Nginx kontejner montira nginx.conf fajl kao read-only volum i zavisi od svih backend servisa.

Korisne komande za upravljanje Docker Compose okruženjem: **docker compose up --build** pokreće kompletan sistem sa izgradnjom svih slika; **docker compose up --build --no-deps -d <service-name>** obnavlja i pokreće jedan servis bez restartovanja zavisnih kontejnera — ovo je najkorisnija komanda tokom razvoja kada se menja jedan servis; **docker compose down -v** zaustavlja sistem i briše sve volume-ove čime se baze podataka resetuju na inicijalno stanje; **docker compose logs -f <service-name>** prati log ispis konkretnog servisa u realnom vremenu.

13.3. CI/CD pipeline — GitHub Actions

Projekat koristi GitHub Actions za Continuous Integration, definisan u *.github/workflows/ci.yml*. Pipeline se aktivira pri svakom push na main ili dev granu, kao i pri otvaranju Pull Request-a ka tim granama.

Ključna karakteristika pipeline-a je inteligentna detekcija promena putem dorny/paths-filter@v3 akcije. Prvi job, detect-changes, analizira modificirane putanje i za svaki servis generiše boolean output (department, goal, premise, gqm-goal, assessment, user, orchestration, audit, shared, infrastructure). Ovi output-i koriste se u job-ovima koji slede.

Za svaki od osam servisa definisan je zaseban build job. Svaki job pokreće se jedino ako je detektovana promena u tom servisu ili u shared/ direktorijumu — promena u deljenim bibliotekama aktivira build svih servisa jer sve od njih zavise. Koraci svakog build job-a su uniformni: preuzimanje koda (actions/checkout@v4), podešavanje .NET 10 SDK (actions/setup-dotnet@v4), NuGet keširanje za ubrzanje narednih build-ova, izgradnja zavisnih deljenih biblioteka, izgradnja servisa u Release konfiguraciji, pokretanje test projekta sa generisanjem TRX izveštaja i objavljivanje test rezultata na GitHub Actions UI putem dorny/test-reporter@v1.

Svi build job-ovi izvršavaju se paralelno jer nemaju međusobnih zavisnosti osim prema detect-changes job-u. Docker build korak je komentaran u konfiguraciji i predviđen je kao naredna faza (CD pipeline) koja bi obuhvatila izgradnju i objavljivanje Docker slika.

13.4. Strategija grananja i verziona kontrola

Projekat koristi trostepenu Git strategiju grananja: **main** grana sadrži uvek stabilan, produkcijski spreman kod. **dev** grana je integraciona grana na kojoj se spajaju dovršene funkcionalnosti. **feature/{feature-name}** grane su kratkotrajne radne grane na kojima se razvijaju pojedinačne funkcionalnosti.

Tok rada je sledeći: programer kreira feature granu od dev, razvija i commituje promene, otvara Pull Request ka dev grani koji aktivira CI pipeline, a po uspešnom prolasku ci proverava i pregledu koda, grana se spaja u dev. Kada je dev grana stabilna i temeljno testirana, pravi se PR ka main koji se spaja i time se zvanično objavljuje nova verzija sistema.

13.5. Prometheus

Kao primarni alat za prikupljanje metrika aplikacija koristi Prometheus. On se pokreće u obliku kontejnera mapiranog sa specifičnom konfiguracijskom `prometheus.yml` datotekom i skuplja mrežne podatke periodično proveravajući `/health` i ostale infrastrukturne interfejsse svih osam backend servisa. Zahvaljujući njemu, tim u svakom trenutku može analizirati potrošnju procesora, popunjenost memorije, i ukupan broj grešaka i HTTP statusa iz celog distribuiranog ekosistema kako unutar klastera tako i van njega.

13.6. Grafana

Nadovezujući se na podatke prikupljene kroz Prometheus bazu, Grafana se integriše kao platforma za kreiranje informativnih i dinamičnih grafičkih dashboard-a. Konfigurisana je tako da pri pokretanju iz observability foldera automatski podigne već unapred definisane šablone za praćenje i učitava Prometheus kao "data source". Zahvaljujući naprednom setu panela omogućena je vizuelna inspekcija performansi u realnom vremenu uz pravovremeno dobijanje upozorenja (alerting), što znatno olakšava proces nadzora i dijagnostiku problema.

13.7. Jaeger

S obzirom na to da obrada jednog inicijalnog korisničkog zahteva gotovo redovno obuhvata i komunikaciju između većeg broja servisa, platforma se oslanja na OpenTelemetry tehnologiju i Jaeger alat za takozvano distribuirano praćenje (tracing). Podaci o obradi HTTP zahteva zajedno sa odgovarajućim korelacijskim identifikatorima prikupljaju se bez prekida i dostavljaju se Jaeger-u. Putem Jaeger UI panela developeri mogu detaljno da ispituju vizuelni put mrežnih zahteva, otkriju tačku prekida ili vremensko usko grlo u sistemu, obezbeđujući lakšu optimizaciju.

14. Testiranje

14.1. Strategija testiranja

Svaki od osam mikroservisa poseduje zaseban test projekat (`{ServisName}.Tests`) koji sadrži automatizovane testove pisane korišćenjem xUnit test framework-a, koji je industrijski standard za .NET testove.

Testovi su organizovani prema tipu: unit testovi za poslovnu logiku servisa i integracioni testovi za specifične komponente. Pristup testiranja naglašava izolaciju — testira se samo jedan sloj bez oslanjanja na stvarne infrastrukturne komponente. Application servisni testovi mockuju DbContext interfejsse i HTTP klijente, čime se poslovna pravila testiraju u potpunoj izolaciji od baze podataka i mreže.

14.2. Unit testovi servisa

Assessment Service poseduje AssessmentServiceTests klasu koja testira poslovnu logiku servisa za procene verovatnoće — kreiranje, ažuriranje i dohvaćanje procena uz verifikaciju poslovnih pravila kao što je validacija opsega verovatnoće. Goal Service ima testove koji pokrivaju upravljanje ciljevima i poslovne pravila tranzicije statusa.

14.3. Testovi validatora

FluentValidation validatori se testiraju nezavisno od servisa, što odražava princip da validaciona pravila čine zasebnu testiranu jedinicu. CreateAssessmentValidatorTests testira sve granične slučajeve validacije ulaznog DTO-a za kreiranje procene: neispravne opsege verovatnoće, prazna obavezna polja i slično. Ovakav pristup daje visok stepen poverenja u ispravnost validacione logike bez potrebe za pokretanjem čitave pipeline-e.

14.4. Testovi domenskih komponenti

Audit Service sadrži AuditLogNotFoundExceptionTests koji testira da domenski izuzetak AuditLogNotFoundException pravilno inicijalizuje poruku greške. Ovaj tip testa osigurava ispravno ponašanje domenskih izuzetaka koji se koriste u GlobalExceptionHandlerMiddleware za generisanje odgovarajućih HTTP odgovora.

14.5. E2E testiranje

Pored unit testova i domenskih testova o kojima je bilo reči, platforma sadrži i sveobuhvatne E2E (End-to-End) testove unutar E2E.Tests projekta. Kroz E2E automatizovane testove proveravaju se realni korisnički scenariji prolazeći kroz celokupan sistem – od Nginx API gateway-a, preko različitih mikroservisa sve do baza podataka i RabbitMQ redova za poruke. Ovaj vid testiranja na visokom nivou pruža potpunu garanciju da delovi sistema mogu međusobno usklađeno interagovati i da aplikacija kao jedinstvena celina u potpunosti zadovoljava definisane poslovne zahteve pre deploymenta.

14.6. Automatizovano pokretanje u CI

GitHub Actions CI pipeline automatski pokreće test projekte za sve servise koji su doživeli promene koristeći dotnet test --configuration Release --logger trx --results-directory test-results. TRX format je XML-baziran format izveštaja o testovima kojeg GitHub Actions može prikazati u UI. dorny/test-reporter@v1 akcija čita TRX fajlove i prikazuje rezultate direktno na stranici pull request-a ili github actions run-a, sa detaljima o uspešnim i neuspešnim testovima. Opcija if: always() osigurava da se rezultati publikuju čak i ako testovi ne prođu, što je važno za dijagnostiku neuspelih build-ova.

15. Zaključak

Sistem GqmPlus realizovan je kao savremena, distribuirana platforma koja u potpunosti implementira GQM+ Strategies metodologiju upravljanja strateškim ciljevima. Implementacija mikroservisne arhitekture sa osam nezavisnih servisa, svaki sa sopstvenom PostgreSQL bazom podataka, osigurava visok stepen izolacije, nezavisnu rasporedivost i horizontalnu skalabilnost sistema. Primena čiste slojevite arhitekture unutar svakog servisa osigurava dugotrajnu održivost i lakoću proširenja.

Bezbednosni model kombinuje JWT autentifikaciju za korisničke sesije i HMAC potpise za međuservisnu komunikaciju, čime je sistem zaštićen na svakom nivou. Korelacioni ID mehanizam i centralizovani audit log osiguravaju punu revizijsku vidljivost svih operacija kroz distribuirani sistem. Saga obrazac u Orchestration servisu brine o konzistentnosti podataka u situacijama kada više servisa mora biti koordinisano.

Angular 21 frontend sa standalone komponentama, lazy loading rutiranjem, granularnim permission sistemom i reaktivnim upravljanjem stanjem pruža korisnicima intuitivno i bezbedno sučelje. Konfiguracija komunikacije uvek prolazi kroz Nginx API Gateway koji centralizuje ulazak u sistem i osigurava uniformno propagiranje važnih zaglavlja.

Infrastrukturna postavka zasnovana na Docker Compose-u i GitHub Actions CI pipeline-u omogućava reproducibilno postavljanje sistema i automatizovanu verifikaciju ispravnosti koda pri svakoj izmeni. Inteligentna detekcija promena u CI pipeline-u osigurava da se grade i testiraju samo delovi koji su modifikovani, čime se vreme izvršavanja pipeline-a minimizuje.

Projekat kao celina demonstrira primenu industrijsko-relevantnih obrazaca i tehnologija u realizaciji složenog poslovnog sistema: database-per-service, CQRS-similar layering, Saga orkestracija, JWT+HMAC bezbednosni model, IHttpConnectionFactory sa delegating handler lancima, lazy loading Angular aplikacija i path-based CI/CD.